Output Reference

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Naming convention . . . 2
Per-group pages . . . 3
Output index file . . . 3
Filename grammar . . . 3
Scopes . . . 4
Groups . . . 4
Intervals . . . 4
print.prt toggle . . . 5
Subdaily output . . . 5
Always-on outputs . . . 5
Purpose . . . 5
Source . . . 5
Format . . . 6
Aggregation . . . 7
Example . . . 7
Related files . . . 7
Purpose . . . 7
Legacy channel files . . . 8
sd_channel files . . . 9
Channel sediment budget and morphology . . . 10
Related files . . . 11
Aquifer . . . 11
Reservoir . . . 12
Crop yield . . . 12
Plant and weather . . . 13
Related files . . . 14
Purpose . . . 14
Source . . . 14
File naming . . . 14
Per-family print.prt gating . . . 15
Legacy CSU carbon outputs . . . 15
HRU files . . . 16
LSU files . . . 18
Optional supporting file . . . 18
Related pages . . . 19
Purpose . . . 19
Source . . . 19
Output files . . . 19
print.prt toggles . . . 21
Related pages . . . 21
simulation.out . . . 21
diagnostics.out . . . 22
checker.out . . . 23
files_out.out . . . 23

erosion.out . . . 24
success.fin . . . 24
erosion.txt . . . 24
Recommended checks after every run . . . 24
Related files . . . 24
test/spcheck.py . . . 24
test/swat_output_to_csv_files.py . . . 25
Related files . . . 27

SWAT+ writes one text file per (scope, group, interval) combination. Every output is toggled in `print.prt`. This section documents the file groups, their columns, and where they are written in the source.

Naming convention

All time-step output files follow the pattern:

`<scope>_<group>_<interval>.txt`

When `csvout = y` in `print.prt`, a matching `.csv` variant is also written.

Scopes (the spatial aggregation level):

Scope	Meaning
`hru_`	one row per Hydrologic Response Unit
`hru-lte_`	one row per low-resolution HRU (lite model)
`lsunit_`	one row per landscape unit (`ls_unit.def`)
`ru_`	one row per routing unit
`region_`	one row per region defined for regional output
`basin_`	one row, the basin-wide sum or average
`channel_`	one row per channel object (legacy `cha`)
`channel_sd` / `sd_chan`	one row per channel object (sd_channel routing)
`aquifer_`	one row per aquifer object
`reservoir_`	one row per reservoir object
`wetland_`	one row per wetland object
`recall_`	one row per recall object (point source or inlet)
`crop_yld_`	one row per HRU per harvested plant

Groups (the variable family):

Group	Meaning
`_wb`	water balance (precipitation, ET, runoff, percolation, storage)
`_nb`	nutrient balance (mineral and organic N and P pools)
`_ls`	losses from the landscape (sediment, nutrient yields)
`_pw`	plant and weather (LAI, biomass, yield, temperature, stress)
`_cha`	channel routing (legacy: flow, sediment, nutrients)
`_sd_cha`	sd_channel routing (water body storage, in, out)
`_aqu`	aquifer storage and fluxes
`_res`	reservoir storage and fluxes
`_pest`	pesticide
`_salt`	salt ions
`_cs`	constituents
`_carbon`	soil carbon pools
`_chanbud`	channel sediment budget
`_chamorph`	channel morphology

Intervals (the time aggregation):

Suffix	Meaning
`_day`	one row per simulated day
`_mon`	one row per month
`_yr`	one row per year
`_aa`	one row, average annual over the print window

Each interval is enabled independently in `print.prt`. A row in `print.prt` named `hru_wb` with `y` in the `avann` column produces `hru_wb_aa.txt`; with `y` in the `daily` column it also produces `hru_wb_day.txt`. The file is opened in `output_landscape_init.f90` only if the matching switch is `y`.

Per-group pages

- Naming convention. The full grammar and the `print.prt` mapping.
- Water balance. The `*_wb_*.txt` columns: precipitation, ET, runoff, percolation, soil and snow storage.
- Channel and sediment. The legacy `channel_*.txt` and the sd_channel `channel_sd_*.txt` columns, including sediment and nutrient routing.
- Aquifer, reservoir, plant. The `aquifer_*.txt`, `reservoir_*.txt`, `crop_yld_*.txt`, and plant/water-stress (`*_pw_*.txt`) columns.
- Checker and diagnostics. The always-on diagnostic files: `checker.out`, `diagnostics.out`, `simulation.out`, `files_out.out`.
- gwflow. 2D groundwater outputs: basin and per-cell water balance, heat, six solute species, observation wells, and pumping.
- Post-processing helpers. The two Python scripts shipped in `test/` for regression checks and CSV conversion.

Output index file

When SWAT+ runs, it writes a master index `files_out.out` listing every output file actually opened during that run. The first column is a short tag (`HRU`, `BASIN`, `CHK`, `CHN`, `AQU`, `RES`, …) and the second column is the filename. Open `files_out.out` first to see exactly which outputs your `print.prt` produced.

Filename grammar

`<scope>_<group>_<interval>.<ext>`

- `scope` identifies the spatial unit.
- `group` identifies the variable family.
- `interval` identifies the time aggregation.
- `ext` is `txt` (always) and `csv` (when `csvout = y` in `print.prt`).

Examples that the source actually opens:

```
hru_wb_day.txt
hru_wb_mon.txt
hru_wb_yr.txt
hru_wb_aa.txt
basin_wb_aa.txt
lsunit_wb_aa.txt
hru-lte_wb_aa.txt
hru_nb_aa.txt
hru_ls_aa.txt
hru_pw_aa.txt
basin_aqu_aa.txt
basin_res_aa.txt
basin_cha_aa.txt
basin_sd_cha_aa.txt
```

channel_sd_day.txt
channel_day.txt
aquifer_day.txt
reservoir_day.txt
crop_yld_aa.txt
crop_yld_yr.txt

Scopes

The scope prefix maps to a writer subroutine:

Scope	Writer	Loop body
hru_	hru_output.f90	one record per HRU
hru-lte_	hru_lte_output.f90	one record per HRU-LTE
lsunit_	lsu_output.f90	one record per landscape unit
ru_	ru_output.f90	one record per routing unit
region_	lsreg_output.f90	one record per region per landuse
basin_	basin_output.f90, basin_aquifer_output.f90, basin_channel_output.f90, basin_sdchannel_output.f90, basin_reservoir_output.f90	one record (basin sum or average)
channel_	channel_output.f90	one record per legacy channel object
channel_sd (file group sd_cha)	sd_channel_output.f90	one record per sd_channel object
aquifer_	aquifer_output.f90	one record per aquifer
reservoir_	reservoir_output.f90	one record per reservoir
wetland_	wetland_output.f90	one record per wetland
recall_	recall_output.f90	one record per recall object
crop_yld_	hru_output.f90 (last block)	one record per HRU per plant

Groups

The group token identifies the column family. Each writer holds its own Fortran derived type and the columns are the fields of that type in declaration order.

Group	Type	Module
_wb	output_waterbal	src/output_landscape_module.f90
_nb	output_nutbal	src/output_landscape_module.f90
_ls	output_losses	src/output_landscape_module.f90
_pw	output_plantweather	src/output_landscape_module.f90
_cha	ch_output	src/channel_module.f90
_sd_cha	water_body plus two hyd_output records (in, out)	src/water_body_module.f90, src/hydrograph_module.f90
_aqu	aquifer_dynamic	src/aquifer_module.f90
_res	water_body plus two hyd_output records (in, out)	src/water_body_module.f90, src/hydrograph_module.f90
_pest	pesticide processes types	src/output_ls_pesticide_module.f90, others
_salt	salt ion types	src/salt_module.f90
_cs	constituents types	src/cs_module.f90
_carbon, _soilcarb, _rescarb, _plcarb	carbon pool types	src/output_landscape_module.f90, src/carbon_module.f90

Intervals

Suffix	Meaning	print.prt column
_day	one row per day	daily
_mon	one row per month	monthly
_yr	one row per year	yearly
_aa	one row, average annual over the print window	avann

The print window is the simulation range minus nyskip warm-up years (set in the first table of print.prt).

print.prt toggle

print.prt has a table of object groups, one row per group, with four columns (daily, monthly, yearly, avann). Each cell is y or n. The file is opened only if the cell is y. The toggles set pco%<group>%<d|m|y|a> (read in print_read.f90) and the file is opened in output_landscape_init.f90 or one of the header_*.f90 files.

Example: in refdata/Ames_sub1/print.prt, the row

```
hru_wb                          n              n              n              y
```

opens hru_wb_aa.txt only. The other three (day, mon, yr) are not written.

A separate row, csvout, switches on the matching .csv variant for every enabled file.

crop_yld in print.prt accepts n, a (annual only), y (yearly only), or b (both). Other groups use the four-column form.

Subdaily output

When time%step > 1 (subdaily simulation), additional *_subday.txt files are written. The current subdaily channel writer is in sd_channel_output.f90 (channel_sd_subday.txt, unit 2508).

Always-on outputs

A few files are not controlled by print.prt and are written every run:

- simulation.out. Daily progress log written from main.f90 and time_control.f90.
- files_out.out. Index of every output file opened, written from proc_bsn.f90.
- diagnostics.out. Warnings about missing input references, opened in proc_bsn.f90.
- checker.out. HRU-level parameter sanity dump, written in proc_hru.f90.
- erosion.out. USLE inputs and outputs per HRU.
- success.fin. Empty marker created on successful completion.

See Checker and diagnostics for the column-level breakdown.

Purpose

Water balance files report the major flows and storages of the HRU-to-channel hydrology: precipitation, snow, ET, surface runoff, lateral and tile flow, percolation, soil-water storage, and snow pack. One file is written per scope (hru, hru-lte, lsunit, ru, region, basin) per interval (day, mon, yr, aa), toggled in print.prt.

Source

- HRU writer: src/hru_output.f90, units 2000 (day), 2001 (mon), 2002 (yr), 2003 (aa).
- Landscape unit writer: src/lsu_output.f90.
- Routing unit writer: src/ru_output.f90.
- Basin writer: src/basin_output.f90.
- Region writer: src/lsreg_output.f90.

- File opening and header writes: src/output_landscape_init.f90.
- Record type: src/output_landscape_module.f90, `type output_waterbal`.

Format

- Line 1: title (project name, version banner).
- Line 2: column header (`wb_hdr`).
- Line 3: units (`wb_hdr_units`).
- Lines 4+: data records. One record per object (HRU, lsunit, …) per timestep.

The leading seven columns are the same for every landscape output:

#	Column	Type	Description
1	jday	int	Julian day of year
2	mon	int	calendar month
3	day	int	day of month
4	yr	int	calendar year
5	unit	int	spatial id (HRU number, lsunit number, …)
6	gis_id	int	GIS id from the object table
7	name	char	object name (`ob(iob)%name`)

The remaining columns come from `output_waterbal`, in declaration order. All flows are mm of water depth over the unit area; CN is dimensionless.

#	Column	Units	Description
8	precip	mm	precipitation (rain + snow)
9	snofall	mm	precipitation falling as snow, sleet, or freezing rain
10	snomlt	mm	snow or melting ice
11	surq_gen	mm	surface runoff generated on the landscape
12	latq	mm	lateral soil flow
13	wateryld	mm	water yield (surq + latq + tile)
14	perc	mm	percolation out of soil profile into vadose zone
15	et	mm	actual evapotranspiration
16	ecanopy	mm	canopy evaporation (not reported in this column in current source)
17	eplant	mm	plant transpiration
18	esoil	mm	soil evaporation
19	surq_cont	mm	surface runoff leaving the landscape
20	cn	none	average curve number for the timestep
21	sw_init	mm	soil water content at start of timestep
22	sw_final	mm	soil water content at end of timestep
23	sw_ave	mm	average soil water content over the timestep
24	sw_300	mm	final soil water content in upper 300 mm
25	sno_init	mm	snow water equivalent at start of timestep
26	sno_final	mm	snow water equivalent at end of timestep
27	snopack	mm	average snow water equivalent over the timestep
28	pet	mm	potential evapotranspiration
29	qtile	mm	subsurface tile flow leaving the landscape
30	irr	mm	irrigation water applied
31	surq_runon	mm	surface runoff received from upland landscape
32	latq_runon	mm	lateral flow received from upland landscape
33	overbank	mm	overbank flooding from channels
34	surq_cha	mm	surface runoff flowing into channels
35	surq_res	mm	surface runoff flowing into reservoirs
36	surq_ls	mm	surface runoff flowing onto a landscape element
37	latq_cha	mm	lateral flow into channels
38	latq_res	mm	lateral flow into reservoirs
39	latq_ls	mm	lateral flow into a landscape element

#	Column	Units	Description
40	gwsoilq	mm	groundwater transferred to soil profile (gwflow)
41	satex	mm	saturation excess flow developed from high water table (gwflow)
42	satex_chan	mm	saturation excess flow reaching the main channel (gwflow)
43	sw_change (delsw)	mm	change in soil water volume
44	lagsurf	mm	surface runoff in transit to channel
45	laglatq	mm	lateral flow in transit to channel
46	lagsatex	mm	saturation excess flow in transit to channel
47	wet_evap	mm	evaporation from wetland surface
48	wet_out (wet_oflo)	mm	outflow (spill) from wetland
49	wet_stor	mm	wetland storage at end of timestep

The HRU writer also appends two trailing string fields after the numeric columns, written by `hru_output.f90`:

Trailing	Description
plant_cov	name of the plant community from `lum.dtl`
mgt_ops	name of the management schedule from `lum.dtl`

Aggregation

- Daily (`_day`): values for the day. Files use unit 2000 (HRU), 2050 (basin), 2140 (lsunit).
- Monthly (`_mon`): daily values summed; `sw`, `sno`, and `cn` reset their initial/final markers at month boundaries. Unit 2001 (HRU), 2051 (basin), 2141 (lsunit).
- Yearly (`_yr`): monthly values summed; storage variables divided by 12 where appropriate. Unit 2002 (HRU), 2052 (basin), 2142 (lsunit).
- Annual average (`_aa`): yearly values divided by `time%yrs_prt`. Unit 2003 (HRU), 2053 (basin), 2143 (lsunit).

Example

`refdata/Ames_sub1/basin_wb_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
  jday    mon    day     yr     unit  gis_id  name                  precip     snofall      snomlt    surq_gen
                                                                       mm          mm          mm          mm
   366     12     31   2020          1       1  demo                1905.214     220.690     219.997      59.5
```

The 42 numeric columns after `name` map one-to-one onto fields 8 to 49 above. The trailing `Original Simulation` token is the scenario name (basin scope substitutes it for the HRU `plant_cov`/`mgt_ops` pair).

Related files

- Naming convention.
- Channel and sediment outputs.
- `print.prt`. Toggles each file group on or off.

Purpose

SWAT+ supports two channel-routing implementations, each with its own output set.

- **Legacy channel** (`channel_*.txt`, `basin_cha_*.txt`). One row per channel object, holding inflow / outflow of water, sediment, sediment by particle size, nutrients, pesticides, bacteria, and conservative metals.

- **sd_channel** (`channel_sd_*.txt`, `basin_sd_cha_*.txt`). Storage-and-discharge routing. Three blocks of columns: a `water_body` block (area, precip, evap, seep, storage), followed by an inflow `hyd_output` record and an outflow `hyd_output` record.

The channel sediment budget and morphology have their own writers (`sd_chanbud_output.f90`, `sd_chanmorph_output.f90`) and produce `*_chanbud_*.txt` and `*_chamorph_*.txt`.

Legacy channel files

Source

- Writer: src/channel_output.f90, units 2480 (day), 2481 (mon), 2482 (yr), 2483 (aa).
- Basin sum: src/basin_channel_output.f90.
- File opening: src/header_channel.f90.
- Record type: src/channel_module.f90, `type ch_output`.

Format

Each line is the seven leading columns (jday, mon, day, yr, unit, gis_id, name), then the 60 fields of `ch_output` in declaration order.

#	Column	Units	Description
1	flo_in	ha-m	streamflow into reach
2	flo_out	ha-m	streamflow out of reach
3	evap	m^3/s	rate of water loss by evaporation
4	tloss	m^3/s	rate of transmission loss through streambed
5	sed_in	tons	sediment transported into reach
6	sed_out	tons	sediment transported out of reach
7	sed_conc	mg/L	sediment concentration in reach
8	orgn_in	kg N	organic nitrogen in
9	orgn_out	kg N	organic nitrogen out
10	orgp_in	kg P	organic phosphorus in
11	orgp_out	kg P	organic phosphorus out
12	no3_in	kg N	nitrate in
13	no3_out	kg N	nitrate out
14	nh4_in	kg	ammonium in
15	nh4_out	kg	ammonium out
16	no2_in	kg	nitrite in
17	no2_out	kg	nitrite out
18	solp_in	kg P	soluble phosphorus in
19	solp_out	kg P	soluble phosphorus out
20	chla_in	kg	chlorophyll-a in
21	chla_out	kg	chlorophyll-a out
22	cbod_in	kg	carbonaceous BOD in
23	cbod_out	kg	carbonaceous BOD out
24	dis_in	kg	dissolved oxygen in
25	dis_out	kg	dissolved oxygen out
26	solpst_in	mg pst	soluble pesticide in
27	solpst_out	mg pst	soluble pesticide out
28	sorbpst_in	mg pst	sorbed pesticide in
29	sorbpst_out	mg pst	sorbed pesticide out
30	react	mg pst	pesticide loss by reaction in water
31	volat	mg	pesticide loss by volatilization
32	setlpst	mg pst	pesticide settling from water to bed
33	resuspst	mg	pesticide resuspended from bed
34	difus	mg	pesticide diffused from water to bed
35	reactb	mg	pesticide reaction loss in bed
36	bury	mg	pesticide buried
37	sedpest	mg	pesticide in bed sediment
38	bacp	# cfu/100mL	persistent bacteria out

#	Column	Units	Description
39	baclp	# cfu/100mL	less persistent bacteria out
40	met1	kg	conservative metal #1 out
41	met2	kg	conservative metal #2 out
42	met3	kg	conservative metal #3 out
43	sand_in	tons	sand in
44	sand_out	tons	sand out
45	silt_in	tons	silt in
46	silt_out	tons	silt out
47	clay_in	tons	clay in
48	clay_out	tons	clay out
49	smag_in	tons	small aggregate in
50	smag_out	tons	small aggregate out
51	lag_in	tons	large aggregate in
52	lag_out	tons	large aggregate out
53	grvl_in	tons	gravel in
54	grvl_out	tons	gravel out
55	bnk_ero	tons	bank erosion
56	ch_deg	tons	channel degradation
57	ch_dep	tons	channel deposition
58	fp_dep	tons	floodplain deposition
59	tot_ssed	mg/L	total suspended sediments

Example header

`refdata/Ames_sub1/basin_cha_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
  jday     mon    day    yr     unit  gis_id  name                  flo_in        flo_out             evap
                                                                      ha-m           ha-m             ha-m
```

The header reports `evap` and `tloss` units as `ha-m`, but `ch_output` carries them as `m^3/s` (rate). The unit row in the file does not match the source comment.

sd_channel files

Source

- Writer: src/sd_channel_output.f90, units 2500 (day), 2501 (mon), 2502 (yr), 2503 (aa); subdaily on unit 2508.
- Basin sum: src/basin_sdchannel_output.f90.
- File opening: src/header_sd_channel.f90.
- Record types: src/water_body_module.f90, `type water_body`, and src/hydrograph_module.f90, `type hyd_output`.

Format

Each record is the seven leading columns, then four blocks of fields written in this order by `sd_channel_output.f90`:

```
write (2500,100) time%day, time%mo, time%day_mo, time%yrc, ichan, ob(iob)%gis_id, ob(iob)%name,
 ↪ &
  ch_wat_d(ichan)%area_ha, ch_wat_d(ichan)%precip, ch_wat_d(ichan)%evap, ch_wat_d(ichan)%seep,
 ↪ &
  ch_stor(ichan), ch_in_d(ichan), ch_out_d(ichan), wtemp
```

Block 1: water body status (4 fields).

#	Column	Units	Description
1	area	ha	channel water surface area
2	precip	m^3	precipitation on water surface
3	evap	m^3	evaporation from water surface
4	seep	m^3	seepage from streambed

Block 2: channel storage `ch_stor` (one `hyd_output` record, 18 fields). These are the `_stor` columns in the file header.

Block 3: inflow `ch_in_d` (one `hyd_output` record, 18 fields). These are the `_in` columns.

Block 4: outflow `ch_out_d` (one `hyd_output` record, 18 fields). These are the `_out` columns.

Block 5: water temperature `wtemp` (deg C), one trailing scalar.

The 18 fields of `hyd_output`, in declaration order:

Field	Units	Description
flo	m^3 (storage), m^3/s (in / out flow rate)	volume or flow rate
sed	metric tons	sediment
orgn	kg N	organic N
sedp	kg P	organic P
no3	kg N	NO3-N
solp	kg P	soluble P
chla	kg	chlorophyll-a
nh3	kg N	NH3
no2	kg N	NO2
cbod	kg	carbonaceous BOD
dox	kg	dissolved oxygen
san	tons	detached sand
sil	tons	detached silt
cla	tons	detached clay
sag	tons	detached small aggregate
lag	tons	detached large aggregate
grv	tons	gravel
temp	deg C	temperature

The line ends with `wtemp` after the `out` block.

Example header

`refdata/Ames_sub1/basin_sd_cha_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
   jday   mon   day    yr     unit   gis_id   name                    area         precip           evap
                                                                       ha            m^3            m^3
```

Each block ends with a `null`-labelled column. That column is the `temp` field of `hyd_output`. The writer passes `ch_in_d(ichan)`, `ch_out_d(ichan)`, and `ch_stor(ichan)` as whole records, so the 18th field of each block is `temp`. The basin-wide water temperature is the trailing `wtemp` scalar written after the outflow block.

Channel sediment budget and morphology

- `sd_chanbud_*.txt`. Sediment budget per channel: bank erosion, bed erosion, deposition. Writer: `src/sd_chanbud_output.f90`. Basin sum in `src/basin_chanbud_output.f90`.
- `sd_chamorph_*.txt`. Channel morphology: width, depth, slope. Writer: `src/sd_chanmorph_output.f90`. Basin sum in `src/basin_chanmorph_output.f90`.

Related files

- Aquifer, reservoir, plant outputs.
- Water balance outputs.

This page covers four output families:

- Aquifer storage and fluxes (`aquifer_*.txt`, `basin_aqu_*.txt`).
- Reservoir storage and fluxes (`reservoir_*.txt`, `basin_res_*.txt`).
- Crop yields (`crop_yld_aa.txt`, `crop_yld_yr.txt`, `basin_crop_yld_*.txt`).
- Plant and weather (`hru_pw_*.txt`, `lsunit_pw_*.txt`, `basin_pw_*.txt`).

Aquifer

Source

- Writer: `src/aquifer_output.f90`, units 2520 (day), 2521 (mon), 2522 (yr), 2523 (aa).
- Basin sum: `src/basin_aquifer_output.f90`, units 2090-2093.
- File opening: `src/header_aquifer.f90`, `src/header_write.f90`.
- Record type: `src/aquifer_module.f90`, `type aquifer_dynamic`.

Format

Seven leading columns (jday, mon, day, yr, unit, gis_id, name), then the 17 fields of `aquifer_dynamic` in declaration order.

#	Column	Units	Description
1	flo	mm	lateral flow from aquifer
2	dep_wt	m	average depth from surface elevation to water table
3	stor	mm	average water storage in aquifer in timestep
4	rchrg	mm	recharge entering aquifer from other objects
5	seep	mm	seepage from bottom of aquifer
6	revap	mm	plant water uptake plus evaporation from aquifer
7	no3_st	kg/ha N	current total NO3-N mass in aquifer
8	minp	kg/ha P	mineral P transported in return (lateral) flow
9	cbn (header label: `orgn`)	percent	organic carbon in aquifer (static). Header mislabels this column.
10	orgn (header label: `orgp`)	kg/ha N	organic nitrogen in aquifer (static). Header mislabels this column.
11	no3_rchg	kg/ha N	NO3-N flowing into aquifer from another object
12	no3_loss	kg/ha	NO3-N loss
13	no3_lat	kg/ha N	nitrate load to reach in groundwater
14	no3_seep	kg/ha N	nitrate seepage to next object
15	flo_cha	mm	aquifer return flow into channels
16	flo_res	mm	aquifer return flow into reservoirs
17	flo_ls	mm	aquifer return flow into a landscape element

Example header

`refdata/Ames_sub1/basin_aqu_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
   jday   mon   day    yr   unit  gis_id  name                           flo          dep_wt            stor
                                                                          mm               m              mm
```

The `aqu_header` strings in `aquifer_module.f90` declare columns `orgn` (position 9) and `orgp` (position 10), but the `aquifer_dynamic` type fields in those positions are `cbn` (organic carbon percent) and `orgn` (organic nitrogen). The writer pushes whole records in declaration order, so the column under the header `orgn` is carbon and the column under `orgp` is organic nitrogen.

Aggregation

- Daily, monthly, yearly, annual average. Aggregation rules: monthly `stor`, `dep_wt`, `no3_st` are divided by the number of days in the month so they remain averages; yearly divides those three by 12; annual average divides everything by `time%yrs_prt`.

Reservoir

Source

- Writer: src/reservoir_output.f90, units 2540 (day), 2541 (mon), 2542 (yr), 2543 (aa).
- Basin sum: src/basin_reservoir_output.f90.
- File opening: src/header_reservoir.f90.
- Record types: `type water_body` in src/water_body_module.f90, `type hyd_output` in src/hydrograph_module.f90.

Format

The writer call is:

```
write (2540,100) time%day, time%mo, time%day_mo, time%yrc, j, ob(iob)%gis_id, ob(iob)%name, &
  res_wat_d(j), res(j), res_in_d(j), res_out_d(j)
```

Seven leading columns, then four blocks. Each `hyd_output` block contributes its 18 fields. `res(j)` is the full `reservoir` derived type and contributes the storage fields written into the `_stor` columns.

Block	Source	Columns
1	`res_wat_d(j)` (`water_body`)	area (ha), precip (m^3), evap (m^3), seep (m^3)
2	`res(j)` storage as `hyd_output` (`_stor` columns)	18 storage fields: flo, sed, orgn, sedp, no3, solp, chla, nh3, no2, cbod, dox, san, sil, cla, sag, lag, grv, temp
3	`res_in_d(j)` (`hyd_output`, `_in` columns)	18 inflow fields
4	`res_out_d(j)` (`hyd_output`, `_out` columns)	18 outflow fields

See Channel and sediment outputs for the 18 fields of `hyd_output` and their units; the layout is identical to the sd_channel writer (inflow is `m^3/s`, outflow is `m^3/s`, storage is `m^3`).

Example header

`refdata/Ames_sub1/basin_res_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
   jday    mon    day     yr      unit    gis_id    name                      area          precip            evap
```

The basin-level reservoir output uses `m^3` for the inflow / outflow columns instead of the `m^3/s` used by `basin_sd_cha_aa.txt`, because the reservoir code does not divide by seconds. The header reports `m^3` accordingly.

Crop yield

Source

- Writer: bottom block of src/hru_output.f90, units 4008 (aa) and 4010 (yr). Triggered only at `time%end_sim == 1` (for `_aa`) or `time%end_yr == 1` (for `_yr`).
- Basin variant: `basin_crop_yld_*.txt`, written in the basin output path.
- File opening: src/output_landscape_init.f90.

Format

Different shape from the other output files: each line is one HRU per plant, not one HRU per timestep.

#	Column	Units	Description
1	jday	int	last simulated day
2	mon	int	last simulated month
3	day	int	day of month
4	yr	int	year
5	unit	int	HRU number
6	PLANTNM	char	plant name (`pldb(idp)%plantnm`)
7	MASS	kg/ha	total yield mass (dry weight)
8	C	kg/ha	carbon in yield
9	N	kg/ha	nitrogen in yield
10	P	kg/ha	phosphorus in yield

`crop_yld_aa.txt` reports the per-harvest average (yield divided by number of harvests). `crop_yld_yr.txt` reports the yield for that year.

Example

`refdata/Ames_sub1/crop_yld_aa.txt`:

```
 demo
  SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328

                                                                    --YIELD (kg/ha)--
  jday    mon    day     yr     unit PLANTNM                  MASS          C           N           P
   366     12     31   2020        1     corn                                3358.149    1511.167
   366     12     31   2020        1     soyb                                   0.000       0.000
```

Switched on in `print.prt` by the `crop_yld` row (`a` for annual average only, `y` for yearly only, `b` for both).

Plant and weather

Source

- HRU writer: src/hru_output.f90, units 2040 (day), 2041 (mon), 2042 (yr), 2043 (aa).
- File opening: src/output_landscape_init.f90.
- Record type: src/output_landscape_module.f90, `type output_plantweather`.

Format

Seven leading columns, then the 25 fields of `output_plantweather` in declaration order.

#	Column	Units	Description
1	lai	$m^{2/m}2$	leaf area index
2	bioms	kg/ha	total plant biomass
3	yield	kg/ha	harvested biomass yield (dry weight)
4	residue	kg/ha	surface residue cover
5	sol_tmp	deg C	temperature of soil layer 2
6	strsw	days	water stress (drought)
7	strsa	days	excess water stress (aeration)
8	strstmp	days	temperature stress
9	strsn	days	nitrogen stress
10	strsp	days	phosphorus stress
11	strss	days	salinity stress
12	nplnt	kg N/ha	plant N uptake

#	Column	Units	Description
13	percn	kg N/ha	NO3-N leached past soil profile
14	pplnt	kg P/ha	plant P uptake
15	tmx	deg C	maximum air temperature
16	tmn	deg C	minimum air temperature
17	tmpav	deg C	average air temperature
18	solrad	MJ/m^2	solar radiation
19	wndspd	m/s	windspeed
20	rhum	none	relative humidity
21	phubase0	deg C/deg C	base-zero potential heat units
22	lai_max	$m^{2/m}2$	maximum LAI during timestep
23	bm_max	kg/ha	maximum total biomass during timestep
24	bm_grow	kg/ha	total biomass growth during timestep
25	c_gro	kg/ha	total plant carbon growth during timestep

The HRU writer appends `plant_cov` and `mgt_ops` as trailing string fields, just like the water-balance writer.

Aggregation

- Daily values come straight from `hpw_d`.
- Monthly: daily values summed, then divided by days-in-month for the columns that represent averages (`lai`, `tmpav`, etc.), preserving running max for `bm_max` and `lai_max`.
- Yearly: same pattern at year boundary.
- Annual average: divided by `time%yrs_prt` at end of simulation.

Related files

- Water balance outputs.
- Channel and sediment outputs.

Purpose

The carbon output family reports the state and fluxes of the SWAT+ Century-style carbon module: per-HRU and per-LSU gain and loss totals, soil C transformations, per-layer pool composition, plant carbon state, and process drivers. The family supersedes the old sprawl of 47 files in 26 base names. Every file is opt-in through `print.prt`: each carbon family has its own row with the standard four-slot `d / m / y / a` layout.

The outputs are produced only when `bsn_cc%cswat == 2` (the dynamic carbon model; the activation value moved from 1 to 2). See `codes.bsn` and the carbon model theory.

Source

- HRU writer: `src/hru_carbon_output.f90` (gain/loss and transformations) and `src/soil_nutcarb_write.f90` (per-layer pools, fluxes, drivers, dynamics, soil snapshot).
- LSU writer: `src/lsu_carbon_output.f90`.
- File opening and headers: `src/output_landscape_init.f90`.
- Header types and variable-name lists: `src/output_landscape_module.f90` (`output_carb_gl_header`, `output_hscf_header`, …); `src/carbon_module.f90` (`cpool_vars`, `n_p_pool_vars`, `cflux_vars`, `carb_drv_vars`, `carb_dyn_vars`, `soil_snap_vars`).
- Per-family `print.prt` parsing: `src/basin_print_codes_read.f90`, `hru_cb_*` and `lsu_cb_*` case branches.

File naming

Every file follows the canonical convention `<family>_{day,mon,yr,aa}.{txt,csv}`. The `_day` variant is daily, `_mon` is end-of-month, `_yr` is end-of-year, `_aa` is the average-annual record written once at end-of-simulation. The

hru_soil_snap family uses _tot instead of _aa because an annual average of a snapshot pool is degenerate; the _tot file carries one begsim row and one endsim row, so endsim_c - begsim_c is the simulated SOC change per layer (see the carbon outputs reference).

CSV variants are emitted in parallel when csvout = y in print.prt. Both variants share the same column header and units row.

Per-family print.prt gating

Each carbon family has its own row in the per-object output block of print.prt. The basin_print_codes_read.f90 case (...) table accepts these names; each row is one object label followed by four flags (d m y a):

print.prt row	Files gated	Output scope
hru_cb_gl	hru_carb_gl_*.{txt,csv}	HRU
hru_cb_trf	hru_scf_*.{txt,csv}	HRU
hru_cb_lyr	hru_cbn_lyr_*.{txt,csv}	HRU
hru_cb_cpool	hru_cpool_stat_*.{txt,csv}	HRU
hru_cb_npool	hru_n_p_pool_stat_*.{txt,csv}	HRU
hru_cb_plt	hru_plc_stat_*.{txt,csv}	HRU
hru_cb_flux	hru_cflux_stat_*.{txt,csv}	HRU
hru_cb_drv	hru_carb_drv_*.{txt,csv}	HRU
hru_cb_dyn	hru_carb_dyn_*.{txt,csv}	HRU
hru_cb_snap	hru_soil_snap_*.{txt,csv} (uses _tot instead of _aa)	HRU
lsu_cb_gl	lsu_carb_gl_*.{txt,csv}	LSU
lsu_cb_trf	lsu_scf_*.{txt,csv}	LSU
lsu_cb_plt	lsu_plc_stat_*.{txt,csv}	LSU

The output restructure split the previous coarse hru_cb flag into the ten per-family flags above. The old hru_cb and hru_cb_vars rows now drive a parallel set of legacy CSU output files (see below), reinstated so the CSU team's existing workflows keep working alongside the standardised per-family files.

The LSU files are emitted only when db_mx%lsu_out > 0 (the dataset contains routing units), regardless of the lsu_cb_* flag. Datasets without rout_unit.con populated do not create empty LSU carbon files.

Legacy CSU carbon outputs

A second, older set of carbon output files runs alongside the standardised per-family files above. These are not exposed in SWAT+ Editor, so regular users do not see them; the CSU team adds the driving rows to print.prt by hand. This legacy path is temporary and is marked for removal in revision 63.

Two print.prt rows drive it, both read by basin_print_codes_read.f90 and name-driven (a print.prt without them produces none of these files):

print.prt row	When	Files
hru_cb	always when present	hru_cbn_lyr, hru_seq_lyr, hru_n_p_pool_stat
hru_cb = l (layer detail)	flag letter l turns diagnostics on	adds hru_cpool_stat, hru_cflux_stat, hru_plc_stat, hru_begsim_soil_prop, hru_endsim_soil_prop, basin_carbon_all
hru_cb = y (profile)	flag letter y	light set only (diagnostics off)
hru_cb_vars	requires carbon on (cswat == 2)	hru_carbvars, hru_org_allo_vars, hru_org_ratio_vars, hru_org_trans_vars

Each file is written as .txt, plus .csv when csvout = y in print.prt.

The legacy and standardised files hold the same values in a different layout. For example, legacy `hru_cbn_lyr` (totals) plus `hru_seq_lyr` (sequestered) together equal the merged new `hru_cbn_lyr_yr`, which carries totals, sequestered, and depths in one wide table.

The internal `cbn_diagnostics` switch (which gates the heavy diagnostic files) is no longer read from an input file; it is set at startup from the `hru_cb` flag letter (`l` = on, `y` = off). Legacy output uses Fortran file units 8348-8387.

> **Note**
>
> The `cbn_diagnostics` flag in the old `carb_coefs.cbn` is retired. Output gating now lives entirely in `print.prt`.

HRU files

`hru_carb_gl_*` (family `hru_cb_gl`)

Merged HRU carbon gain and loss totals. 21 value columns covering soil C balance, residue C balance, and plant C balance. Replaces the former `hru_soilcarb_*`, `hru_rescarb_*`, and `hru_plcarb_*` triplet. Units are `kg C/ha` for every value column.

Columns (in declaration order in `output_carb_gl_header`): `jday`, `mon`, `day`, `yr`, `unit`, `gis_id`, `name`, then `sed_c`, `surq_c`, `latq_c`, `perc_c`, `res_decay`, `man_app_c`, `man_graze_c`, `rsp_c`, `soil_emit_c`, `plant_surf_c`, `plant_root_c`, `rsd_surfdecay_c`, `rsd_rootdecay_c`, `harv_stov_c`, `rsd_emit_c`, `npp_c`, `harv_abgr_c`, `harv_root_c`, `drop_c`, `grazeat_c`, `plant_emit_c`.

The output restructure broke out four sub-fields that the previous output silently summed under a single label: `plant_surf_c`, `plant_root_c`, `harv_abgr_c`, `harv_root_c`.

`hru_scf_*` (family `hru_cb_trf`)

HRU-level soil carbon transformations (the 13 inter-pool fluxes). One row per HRU per timestep.

Columns: `jday`, `mon`, `day`, `yr`, `unit`, `gis_id`, `name`, then `meta_micr`, `str_micr`, `str_hs`, `co2_meta`, `co2_str`, `micr_hs`, `micr_hp`, `hs_micr`, `hs_hp`, `hp_micr`, `co2_micr`, `co2_hs`, `co2_hp`.

`hru_cbn_lyr_*` (family `hru_cb_lyr`)

Per-layer total soil carbon, written wide (one row per HRU per timestep with per-layer columns). The number of layers in the output is `cb_n_layers`. When the optional `carbon_layers.prt` is absent this defaults to the largest soil layer count across all HRUs, so the deepest profile is written in full; when the file is present it fixes the count to the value given. Each row carries that HRU's own layer depths, and layers a soil does not have are filled with the sentinel `-99`.

Header is built at first data write and carries the structure `layer depth (mm) | total_c_lyr1 | total_c_lyr2 | ... | total_c_lyrN`. Units: `Mg C/ha` per layer column.

`hru_cpool_stat_*` (family `hru_cb_cpool`)

Per-layer C pools. Wide layout, with the columns in `cpool_vars` (`src/carbon_module.f90:20`) expanded to `<var>_lyr1 ... <var>_lyrN`:

```
residue_c, structural_c, metabolic_c, hs_c, hp_c, microbial_c,
lignin_c, nonlignin_c, root_mass, soil_water
```

10 variables x `cb_n_layers` columns. Units are kg C/ha for the C pools, kg/ha for `root_mass`, mm for `soil_water`.

`hru_n_p_pool_stat_*` (family `hru_cb_npool`)

Per-layer N and P content of the carbon pools. Variables in `n_p_pool_vars`:

```
tot_pool_n, residue_n, structural_n, metabolic_n, hs_n, hp_n,
microbial_n, lignin_n, nonlignin_n,
tot_pool_p, residue_p, structural_p, metabolic_p, hs_p, hp_p,
microbial_p, lignin_p, nonlignin_p
```

18 variables x `cb_n_layers` columns. Units kg N/ha or kg P/ha as the column name implies.

`hru_plc_stat_*` (family `hru_cb_plt`)

HRU-level plant carbon state. No per-layer expansion. Seven flat columns:

```
total_c, ab_gr_c, leaf_c, stem_c, seed_c, root_c, surf_rsd_c
```

Units kg C/ha.

`hru_cflux_stat_*` (family `hru_cb_flux`)

Per-layer C and N fluxes from the Century pool transformations. Variables in `cflux_vars` (37 entries):

```
cfmets1, cfstrs1, cfstrs2,                          (C flows from litter)
efmets1, efstrs1, efstrs2,                          (E = effective C)
immmets1, immstrs1, immstrs2,                       (immobilisation)
mnrmets1, mnrstrs1, mnrstrs2,                       (mineralisation)
co2fmet, co2fstr,                                   (CO2 from litter)
cfs1s2, cfs1s3, cfs2s1, cfs2s3, cfs3s1,             (inter-SOC pool C flows)
efs1s2, efs1s3, efs2s1, efs2s3, efs3s1,
imms1s2, imms1s3, imms2s1, imms2s3, imms3s1,
mnrs1s2, mnrs1s3, mnrs2s1, mnrs2s3, mnrs3s1,
co2fs1, co2fs2, co2fs3                              (CO2 from SOC pools)
```

37 variables x `cb_n_layers` columns. Pool indices: `s1` = microbial, `s2` = slow humus, `s3` = passive humus, `met` = metabolic litter, `str` = structural litter.

`hru_carb_drv_*` (family `hru_cb_drv`)

Per-layer environmental drivers and intermediate factors for the Century transformations. Variables in `carb_drv_vars` (14 entries):

```
sut, tillagef, cons_bmix, tillagef_biomix, tillagef_tillmix,
till_eff, cdg, ox, cs, no3, nh4, co2_resp, soil_temp, emix
```

14 variables x `cb_n_layers` columns. `sut`, `cdg`, `ox`, `cs` are dimensionless factors; `no3`, `nh4` are kg/ha; `co2_resp` is kg C/ha; `soil_temp` is deg C; `tillagef*`, `till_eff`, `cons_bmix`, `emix` are dimensionless.

`hru_carb_dyn_*` (family `hru_cb_dyn`)

Per-layer carbon pool dynamics: allocation fractions and the C-to-N ratios of each pool transformation. Variables in `carb_dyn_vars` (21 entries):

```
asp, abp, abco2, a1co2, asco2, apco2,               (CO2 / passive allocations)
ncbm, nchp, nchs,                                   (N:C of microbial, passive, slow)
bmctp, bmntp, hsctp, hsntp, hpctp, hpntp,           (pool C and N totals)
lmctp, lmntp, lsctp, lslctp, lslnctp, lsntp         (litter pool totals)
```

21 variables x `cb_n_layers` columns. Allocation fractions are dimensionless; pool totals are kg C/ha or kg N/ha.

`hru_soil_snap_*` (family `hru_cb_snap`)

Per-layer soil snapshot. Uses `_tot` in place of `_aa` because an annual average of a snapshot is degenerate.

Variables in `soil_snap_vars` (13 entries):

bd, awc, soil_k, tot_c, clay, silt, sand, rock,
alb, usle_k, ec, caco3, ph

13 variables x cb_n_layers columns.

The _day, _mon, _yr files emit end-of-period rows on the usual cadence. The _tot file is special: it emits one row at simulation begin (begsim) and one at simulation end (endsim). Both rows carry pool values in the same unit (kg C/ha for tot_c), so endsim_tot_c_lyrK - begsim_tot_c_lyrK is the simulated SOC change for layer K. The previous design wrote a percentage on the begsim row and a mass on the endsim row, which made the delta meaningless.

LSU files

The LSU files are area-weighted HRU aggregates produced by lsu_carbon_output.f90. Per-layer families are **not** aggregated to LSU because different HRUs within an LSU can have different soil profiles.

lsu_carb_gl_* (family lsu_cb_gl)

LSU-level carbon gain and loss totals, area-weighted from the HRU hru_carb_gl_* values. Reuses the output_carb_gl_header columns (same 21 value columns as the HRU file).

lsu_scf_* (family lsu_cb_trf)

LSU-level soil C transformations, area-weighted from the HRU values. Reuses the output_hscf_header columns.

lsu_plc_stat_* (family lsu_cb_plt)

LSU-level plant carbon, summed across HRUs with the ru_frac weight. Single value column plant_c (kg C/ha), printed once per LSU per timestep.

Optional supporting file

carbon_layers.prt

Optional file that fixes cb_n_layers, the number of soil layers expanded in every wide per-layer carbon file. It is only needed when you want a specific column count; if it is absent the model uses the largest soil layer count across all HRUs (see below). Format:

- Line 1: title (free text, skipped).
- Line 2: header (free text, skipped).
- Line 3: single integer giving the layer count.

Only line 3 carries data. The reader skips lines 1 and 2 entirely, so their text is for the reader's own benefit and may say anything. An integer below 1 is rejected and the automatic default is used instead.

Example

```
carbon_layers.prt: layers to expand in per-layer carbon output
n_layers
10
```

Here line 1 is a free-text title, line 2 is a free-text header, and line 3 is the only value the reader uses. With 10, every per-layer carbon file carries ten layer columns regardless of the soils present, and any unused slots are filled by -99. Use this to force a fixed column count, for example to truncate output to the top layers or to keep a constant width across runs.

If the file is absent, cb_n_layers defaults to the largest soil layer count across all HRUs. This means the deepest profile is written in full and no real layer is truncated; HRUs with fewer layers than that maximum get the sentinel

-99 in their extra slots. (Earlier versions defaulted to a fixed 7, which silently truncated deeper soils.) Reader: src/carbon_layers_read.f90. Not listed in file.cio.

Related pages

- Carbon (model theory). Pool definitions and Century-style transformation equations.
- carbon.bsn. Basin-wide scalar inputs.
- carbon_lyr.bsn. Per-layer inputs.
- print.prt. Output gating.
- Naming convention. Common naming rules across output files.

Purpose

The gwflow module writes its own set of output files when bsn_cc%gwflow = 1 in codes.bsn. Outputs are toggled in print.prt via six rows: gwflow_wb, gwflow_flux, gwflow_heat, gwflow_solute, gwflow_obs, gwflow_pump. Each row has the standard daily/monthly/yearly/average-annual flags.

Source

- Writer: src/gwflow_output.f90
- Types: src/gwflow_module.f90, type groundwater_ss for fluxes
- Simulation-time writes (cell-level): src/gwflow_simulate.f90

Output files

Water balance

File	Scope	Aggregation
gwflow_basin_wb_day.txt	basin total	daily
gwflow_basin_wb_mon.txt	basin total	monthly
gwflow_basin_wb_yr.txt	basin total	yearly
gwflow_basin_wb_aa.txt	basin total	annual average
gwflow_cell_wb_day.txt	per cell	daily
gwflow_cell_wb_mon.txt	per cell	monthly
gwflow_cell_wb_yr.txt	per cell	yearly
gwflow_cell_wb_aa.txt	per cell	annual average

Columns (in declaration order from type groundwater_ss, units m^3):

#	Name	Description
1	chng	change in storage (grid summaries only)
2	rech	recharge
3	gwet	groundwater ET
4	gwsw	groundwater discharge to channels
5	swgw	channel seepage to groundwater
6	satx	saturation excess flow
7	soil	groundwater added to the soil profile
8	latl	lateral flow between cells
9	disp	dispersion (heat and solute transport)
10	bndr	boundary exchange
11	ppag	allocation-driven pumping (irrigation)
12	ppdf	pumping deficit (unmet demand)
13	ppex	external pumping
14	tile	tile drainage outflow

#	Name	Description
15	resv	reservoir exchange
16	wetl	wetland exchange
17	fpln	floodplain exchange
18	canl	canal exchange
19	pond	recharge pond seepage
20	phyt	phreatophyte transpiration
21	totl	sum of inputs and outputs

Per-cell files prepend a cell id and timestamp; basin files aggregate across all cells. The full per-cell water-balance header (`gwflow_cell_wb_*`), verified against `gwflow_output.f90`, is:

```
jday mon day yr unit gis_id name head wt_depth recharge gw_et gw_sw sw_gw sat_excess soil lateral pump_all
```

- `gis_id` is the authoritative cell id carried from the QSWAT+ shapefile; `name` is `cellNNNN` (a 12-character field).
- `head` and `wt_depth` are in metres; the flux columns (`recharge` through `phytorem`) are in m^3/day for daily files (monthly files carry the monthly-average daily rate).
- `wt_depth` = cell land-surface elevation − head (depth of the water table below the cell's representative ground surface). It can look deeper at channel/river cells on coarse grids: the cell surface represents the valley, not the streambed, so the depth is measured from the valley rim while the head sits near the channel. This is a grid-resolution reporting effect, not a model error; the head field itself is correct.

Observation wells

`gwflow_obs_{day,mon,yr,aa}.txt` report state at each observation cell (set via the observation list) for every timestep. Columns:

Column	Units	Meaning
`head`	m	Groundwater head
`wt_depth`	m	Water-table depth below ground (land-surface elevation − head)
`temp`	degC	Groundwater temperature, `-99` when heat transport is off
`no3`	mg/L	Nitrate concentration, `-99` when solute transport is off
`p`	mg/L	Phosphorus concentration, `-99` when solute transport is off

Heat

When the heat transport sub-module is active (`gwflow.heat` is present), the following are written:

File	Aggregation
`gwflow_basin_heat_day.txt`	daily
`gwflow_basin_heat_yr.txt`	yearly
`gwflow_basin_heat_aa.txt`	annual average

Columns are the same fields as water balance, reinterpreted as heat fluxes (the writer reuses `type groundwater_ss`). Refer to the model theory page on Aquifers for the heat transport equations.

Solutes

When solute transport is active (`gwflow.solutes` listing constituents), per-species water-balance-style files are written. Six species are emitted directly by the writer:

Species	Files
Calcium (ca)	gwflow_basin_sol_ca_{day,mon,yr,aa}.txt
Chloride (cl)	gwflow_basin_sol_cl_{day,mon,yr,aa}.txt
Potassium (k)	gwflow_basin_sol_k_{day,mon,yr,aa}.txt
Magnesium (mg)	gwflow_basin_sol_mg_{day,mon,yr,aa}.txt
Sodium (na)	gwflow_basin_sol_na_{day,mon,yr,aa}.txt
Phosphorus (p)	gwflow_basin_sol_p_{day,mon,yr,aa}.txt

Columns follow the same groundwater_ss field set, interpreted as solute mass fluxes (kg).

Cell definition

gwflow_cell_definition.txt is written once at the start of the run. It maps the grid coordinates to cell IDs and is the key for interpreting any per-cell output.

Always-on log

gwflow_record is a plain-text log opened when bsn_cc%gwflow = 1. The simulation banner and per-feature notes are written there. Useful for confirming the gwflow module activated and which sub-features (pumps, ponds, tiles, etc.) loaded.

print.prt toggles

Toggle	Files produced
gwflow_wb	basin and cell water balance
gwflow_flux	gwflow canal, pond, tile, gwsw, channel observation diagnostics
gwflow_heat	basin heat balance
gwflow_solute	basin solute balance for each species
gwflow_obs	observation well output (when gwflow.hru_pump_observe or observation file is set)
gwflow_pump	per-HRU pumping output

The aggregation flags on each row (d, m, y, a) control which suffix variants are written.

Related pages

- gwflow input reference
- codes.bsn (gwflow switch)
- print.prt (output toggles)
- Model theory: Aquifers

These files are written every run, regardless of print.prt. Check them first when validating a simulation.

simulation.out

Source

Opened in src/main.f90 on unit 9003 (first 30 lines). Daily progress lines written from src/time_control.f90. Climate-file open events written from src/proc_date_time.f90.

Content

- Banner: SWAT+ version, revision, compiler, OS, build date.

- One line per climate file as it is opened.
- One line per simulated day:

  `<scenario name> <mon> <day> <year> Yr <yr> of <yr_total> Time hh:mm:ss`
- On successful completion, a line `Execution successfully completed` and a `Date of Sim` stamp.

Example

```
                  SWAT+
             Revision 61.0.1-33-g68b1328
      Soil & Water Assessment Tool
IntelLLVM (2024.0.0), 2024-08-27 08:49:23, Linux
    Program reading . . . executing


  Date of Sim   8/27/2024 Time   8:49:51
 reading from pet file                   Time   8:49:51
 reading from precipitation file         Time   8:49:51
 ...
  Original Simulation        1   1  1975 Yr    1 of   46 Time   8:49:51
  ...
  Original Simulation       12  31  2020 Yr   46 of   46 Time   8:49:53

 Execution successfully completed

  Date of Sim   8/27/2024 Time   8:49:53
```

If `Execution successfully completed` is absent, the run aborted. Search upward in the file for the last day printed: the failure is on the next simulated day.

diagnostics.out

Source

Opened in src/proc_bsn.f90 on unit 9001. Lines are appended by every `*_read.f90` that detects a missing input reference (`grep -rn "write *(9001"` lists about 60 call sites in the source).

Content

One warning per line. Each warning is the name of a missing reference and the input file that pointed to it. Example:

```
 DIAGNOSTICS.OUT FILE
 corn_lum                                CORN_rot                                not found in management.
 agrr_lum                                AGRR_rot                                not found in management.
 soyb_lum                                SOYB_rot                                not found in management.
 ames.tem                                          file not found (tgage)
 file not found (basins_carbon.tes)
```

Read this file end-to-end after every run. Lines here mean SWAT+ silently substituted a default or skipped a setting because the referenced object was not in the database it was supposed to be in. Common categories:

- `<name> not found in management.sch`. A land use management points at a schedule that does not exist.
- `<name> not found (initial.cha)`, `(hydrology.cha)`, `(sediment.cha)`, `(nutrients.cha)`. A channel object points to a property set that does not exist.
- `file not found (<gage>)`. A weather station refers to a gage file that is missing.
- `file not found (<file>)`. A control input (carbon coefficients, snow database) is missing.

An empty `diagnostics.out` (only the header) means no missing references were detected.

checker.out

Source

Opened in src/proc_hru.f90 on unit 4000. Writes one row per HRU after model setup. Header types are `output_checker_header` and `output_checker_unit` in src/hydrograph_module.f90.

Format

- Line 1: scenario name and SWAT+ version.
- Line 2: column header (`chk_hdr`).
- Line 3: unit row (`chk_unit`). The last cell carries the legend `0=notile;1=tile;`.
- Lines 4+: one row per HRU.

#	Column	Units	Description
1	sname	name	soil name
2	hydgrp	A/B/C/D	hydrologic soil group
3	zmx	mm	maximum rooting depth (`soil%zmx`)
4	usle_k	none	USLE soil erodibility factor
5	sumfc	mm	sum of field capacity over profile
6	sumul	mm	sum of saturation (upper limit) over profile
7	usle_p	none	USLE practice factor
8	usle_ls	none	USLE slope-length-steepness factor
9	esco	none	soil evaporation compensation factor
10	epco	none	plant uptake compensation factor
11	cn3_swf	none	curve-number soil-water adjustment
12	perco	none	percolation coefficient
13	latq_co	none	lateral flow coefficient
14	tiledrain	0 or 1	tile drainage flag

Example

`refdata/Ames_sub1/checker.out`:

```
 demo                      SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
                                        zmx       usle_k       sumfc       sumul      usle_p     usle_ls
 sname            hydgrp                (mm)                    (mm)        (mm)
soil_01           B                2000.0000       0.0000     361.5000    616.1781      1.0000      0.3638
soil_02           B                2000.0000       0.0000     361.5000    445.6733      1.0000      0.3638
```

Check `usle_k`, `zmx`, `sumfc`, and `sumul` for unrealistic values (zero, negative, or extreme). A `zmx` of 2000 mm on every HRU usually means the soil definitions were not parsed and the model fell back to a default.

files_out.out

Source

Opened in src/proc_bsn.f90 on unit 9000. Every `open_output_file` call that opens an output writes one line here, naming the file and a short tag.

Format

- Line 1: header `files_out.out - OUTPUT FILES WRITTEN`.
- Lines 2+: two columns, tab/space separated.

Column	Description
tag	`CHK`, `HRU`, `BASIN`, `CHN`, `AQU`, `RES`, `MGT`, …
filename	the file SWAT+ opened

Column	Description

This is the master inventory: anything actually produced by this run is listed here. If you expect a file and it is missing from `files_out.out`, the corresponding switch in `print.prt` was `n`.

erosion.out

Source

Opened in src/proc_hru.f90 on unit 4001. Writes one row per HRU of the USLE inputs and computed factors. Always on; not switchable in `print.prt`.

success.fin

Created in src/main.f90 on unit 107. An empty file written only at the end of a successful run. External tools (calibration drivers, test harnesses) test for its existence to confirm the run completed.

erosion.txt

Opened in `main.f90` on unit 888. Internal debug log of erosion calculations. Not part of the documented output set.

Recommended checks after every run

1. Open `simulation.out` and confirm the last line is `Execution successfully completed`. If not, look at the last printed day.
2. Open `diagnostics.out`. Every line is a problem. Resolve each one before trusting any output.
3. Open `checker.out`. Confirm `zmx`, `sumfc`, `sumul` are sane per HRU. Confirm `tiledrain` flag matches your design.
4. Confirm `success.fin` exists.
5. Open `files_out.out`. Confirm every output file your `print.prt` requested is listed.

Related files

- Naming convention.
- Post-processing helpers. Includes a parser for `checker.out`.

Two Python scripts are bundled in `test/` of the SWAT+ source repository. Each is a small standalone tool. They are not installed by the build; copy them into your working tree or invoke them directly.

test/spcheck.py

A regression-test driver. Runs a SWAT+ executable against a scenario directory, then performs a field-by-field numerical comparison against a golden output.

Subcommands

`spcheck.py` has three subcommands.

run

`spcheck.py run <executable> <scenario>`

Workflow:

1. Looks up `data/<scenario>`.
2. Creates `runs/<scenario>/<NN>#<timestamp>#<executable>/` and copies the scenario into it.
3. Runs `<executable>` (from `build/<executable>`) with that directory as cwd.
4. If the executable returns non-zero, renames the run directory to end in `-FAILED`.

`NN` is the next two-digit serial number (`01`, `02`, …).

compare

`spcheck.py compare <scenario> <a> <b> [--abserr 1e-8] [--relerr 0.05] [--nlines N] [--nerrorlines N]`

Compares the outputs of two prior runs of the same scenario, identified by their serial numbers. Reads the list of output files from `<scenario>/.testfiles.txt` (one filename per line, `#` starts a comment).

For each pair of files, line-by-line, field-by-field:

- Splits the line into tokens by whitespace.
- Skips non-numeric tokens. Skips lines containing `MODULAR` (the version banner).
- Compares each float field: a difference is flagged when

  `abs(f1 - f2) >= aerr + rerr * abs(f2)`
- Defaults: `aerr = 1e-8`, `rerr = 0.05` (5 percent).
- For each diff, prints the line numbers, the field positions, the two values, and the absolute and relative error.

ctest

`spcheck.py ctest <model_path> <data_dir> <tmp_dir> [--abserr ...] [--relerr ...] [--nlines N] [--nerrorlin`

For automated test runs. Copies `<data_dir>` into `<tmp_dir>/<scenario_name>`, runs `<model_path>` in that tmp directory, then compares the newly produced outputs against the originals in `<data_dir>`. Exits non-zero if any comparison errors are above the thresholds.

What it is good for

- Detecting numerical regressions when changing source code.
- Comparing two executables (different compilers, different revisions) against the same scenario.
- Running CI tests with a small abserr/rerr tolerance to flag any unexpected drift.

What it does not do

- Does not understand the column structure of SWAT+ outputs. Tokens are compared by position only.
- Skips integer fields (calls `int()` first; floats only).
- Trusts that both files have the same line count and the same number of float tokens per line. Differences in structure produce a `danger` return code (-1) for the line and the comparison stops on that line.

test/swat_output_to_csv_files.py

A pandas-based parser for SWAT+ text outputs. Maps every known output filename to a read specification (rows to skip, header row, column names, separator), then loads it with `pandas.read_csv`.

How it is structured

A single module-level dictionary `spec_dict` keys every supported filename to a dict of `pandas.read_csv` arguments.

Two default specifications are reused for most files:

```
default_spec = {
    "skiprows": [0, 2],
    "header":   [0],
    "column_names": None,
    "delim_whitespace": True
}

default_spec2 = {
    "skiprows": [0],
    "header":   [0],
    "column_names": None,
    "delim_whitespace": True
}
```

`default_spec` skips line 0 (the SWAT+ banner) and line 2 (the units row), so line 1 becomes the column header. This matches the standard SWAT+ output layout: title, header, units, data.

`default_spec2` keeps the units row in the data. Used for files that do not have a units row (`lu_change_out.txt`, `basin_crop_yld_*.txt`).

`crop_yld_aa.txt` and `crop_yld_yr.txt` get `skiprows=2` (no list) because their banner is two lines long.

`checker.out` and `files_out.out` get explicit `column_names` lists because the file headers do not parse cleanly as a single row of column names.

Files covered

The script supports these outputs out of the box:

```
files_out.out, checker.out
hru_orgc.txt, hru_wb_aa.txt, hru_ncycle_aa.txt, hru_nb_aa.txt,
hru_carbon_aa.txt, hru_nut_carb_gl_aa.txt, hru_ls_aa.txt, hru_pw_aa.txt
basin_wb_aa.txt, basin_nb_aa.txt, basin_carbon_aa.txt,
basin_ls_aa.txt, basin_pw_aa.txt
crop_yld_yr.txt, crop_yld_aa.txt
lu_change_out.txt, basin_crop_yld_yr.txt, basin_crop_yld_aa.txt
hydout_aa.txt, hydin_aa.txt
deposition_aa.txt, wetland_aa.txt
basin_aqu_aa.txt, basin_res_aa.txt
recall_aa.txt
basin_cha_aa.txt, basin_sd_cha_aa.txt, basin_sd_chamorph_aa.txt
basin_psc_aa.txt
ru_aa.txt
```

Entry point

```
def run():
    fname = "files_out.out"
    df = read_output(fname, spec_dict, True)
    for index, row in df.iterrows():
        fname = row["filename"]
        if fname == "mgt_out.txt" or fname == "yield.out":
            continue
        df = read_output(fname, spec_dict, True)
```

When run as `python swat_output_to_csv_files.py`, the script:

1. Reads `files_out.out` from the current directory.
2. For every output filename listed there (other than `mgt_out.txt` and `yield.out`), loads the file with the matching spec and writes a tab-separated `.csv` alongside the original.

`read_output(fname, spec_dict, write_csv=False)` is the unit entry point. Pass `write_csv=True` to write `<fname>.csv`; the function always returns the loaded DataFrame.

Limitations

- The script must be run in the directory containing the SWAT+ outputs.
- If a filename is not in `spec_dict`, the script prints `Swat+ output filename not found in read specification dictionary.` and calls `exit(1)`.
- `pandas.read_csv(..., delim_whitespace=True)` is deprecated in newer pandas versions in favour of `sep=r"\s+"`. Expect a `FutureWarning`.
- The output `.csv` files are tab-separated despite the `.csv` extension (`output_seperator = "\t"`).

Typical usage

```
cd <scenario directory>
python /path/to/swat_output_to_csv_files.py
# produces hru_wb_aa.txt.csv, basin_wb_aa.txt.csv, ... in the same dir
```

Or, in a notebook, import the module and call `read_output` directly on the file of interest to get a DataFrame in memory:

```
import swat_output_to_csv_files as s
df = s.read_output("basin_wb_aa.txt", s.spec_dict, write_csv=False)
```

Related files

- Checker and diagnostics. `files_out.out` is the entry point for the CSV script.
- Naming convention. Explains which file is which.